Q5: Testing and Debugging

** = supplementary, not in the lecture materials · Test types · local vs instrumented · JUnit · Mockito · Espresso · AVD Manager and emulators vs real devices · Logcat · Debugger · Layout Inspector · Profiler

1. Testing vs debugging, and why both are needed

Testing	Debugging
Checks whether the app behaves as expected	**Finds and fixes the cause** of a failure that was observed
Done with test code (JUnit, Espresso) or manually	Done with tools (Logcat, Debugger, Layout Inspector, Profiler)
Answers “does it work?”	Answers “why doesn’t it work?”

Why testing and debugging are required (lecture):

- Mobile apps run on **many device models, Android versions and screen sizes** (fragmentation), so something can work on one phone and fail on another.
- Bugs found **after release are costly** and damage user trust.
- Testing builds **confidence** that the app behaves correctly before shipping.
- Without testing, bugs are found by real users in production. With testing, bugs are caught early with **repeatable** checks.

Design goal: test early, test often, and at the right layer.

2. Types of Android tests

Test type	What it verifies	Where it runs	Tool
Unit test	One function or class **in isolation** from the Android framework	JVM on your computer	JUnit (+ Mockito)
UI test	User-facing behaviour: taps, typing, scrolling, what appears on screen	Emulator or real device	Espresso
Integration test	Several components **working together** (for example DAO + database)	Usually device/emulator	JUnit / AndroidX Test
Manual test	Human testers exploring the app	Device/emulator	None; useful but **not repeatable or scalable**

Key decision: does the check need instant JVM feedback, or a real/emulated Android device? For fast feedback, prefer unit tests; UI tests validate the full experience.

Local (unit) tests vs instrumented tests

Local tests	Instrumented tests
Run on the **JVM** on your machine	Run on an **emulator or real device**
Fast, no device needed	**Slower** to set up and execute
No access to real Android framework APIs	**Access** to the real Android framework and UI
Best for pure logic and calculations	Closer to real app behaviour; for UI and integration
Folder: `src/test`	Folder: `src/androidTest`

Hybrid approach: unit tests cover logic; instrumented tests cover UI and integration. This balances speed and confidence and matches the **testing pyramid**: many fast unit tests at the bottom, fewer integration tests in the middle, fewest slow UI tests at the top.

Project folders and dependencies (practical worksheet)

- `src/test` holds **local tests** (JVM). `src/androidTest` holds **instrumented tests** (need a device). Putting a test in the wrong folder is the **most common reason it refuses to run**.

```
dependencies {
    testImplementation 'junit:junit:4.13.2'                          // local unit tests
    testImplementation 'org.mockito:mockito-core:5.11.0'             // mocking
    androidTestImplementation 'androidx.test.ext:junit:1.1.5'        // AndroidJUnit4 runner
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.5.1'   // UI tests
}
```

`testImplementation` = only for `src/test`. `androidTestImplementation` = only for `src/androidTest`.

3. Unit testing with JUnit

A **unit test** checks that a **single method or class gives the expected output for a given input**. It runs on the JVM without an emulator or device. **JUnit** is the standard framework for writing and running unit tests in Android.

Without unit tests, logic bugs appear only during manual testing or in production. With them, errors are caught **immediately** with fast, repeatable feedback.

The AAA pattern: Arrange, Act, Assert

Step	What it does
Arrange	Set up: create the object and input data
Act	Call the method being tested
Assert	Compare the actual result with the expected result

Class under test and its tests (worksheet + lecture)

```
// src/main/.../Calculator.kt
class Calculator {
    fun add(a: Int, b: Int): Int = a + b
    fun subtract(a: Int, b: Int): Int = a - b
    fun multiply(a: Int, b: Int): Int = a * b
    fun divide(a: Int, b: Int): Int = a / b
}
```

```
// src/test/.../CalculatorTest.kt
import org.junit.Assert.*
import org.junit.Before
import org.junit.Test

class CalculatorTest {
    private lateinit var calculator: Calculator

    @Before                                  // runs before EVERY test
    fun setUp() {
        calculator = Calculator()
    }

    @Test
    fun addition_isCorrect() {
        val result = calculator.add(2, 3)    // Act   (Arrange done in setUp)
        assertEquals(5, result)              // Assert: (expected, actual)
    }

    @Test
    fun subtract_tenMinusFour_returnsSix() {
        assertEquals(6, calculator.subtract(10, 4))
    }

    @Test
    fun multiply_byZero_returnsZero() {
        assertEquals(0, calculator.multiply(0, 99))
    }

    @Test
    fun add_twoNegatives_returnsNegative() {
        assertTrue(calculator.add(-2, -3) < 0)
    }

    @Test
    fun divide_byZero_throwsException() {
        assertThrows(ArithmeticException::class.java) {
            calculator.divide(10, 0)         // integer division by zero throws ArithmeticException
        }
    }
}
```

Test naming style: `methodName_condition_expectedResult` (for example `divide_byZero_throwsException`), so a failing test name tells you exactly what broke.

Why `assertThrows` instead of an empty try/catch? An empty try/catch would pass even if no exception was thrown. `assertThrows` **fails the test** if the expected exception does not happen, and checks the exception type.

JUnit annotations **

Annotation	Meaning
`@Test`	Marks a method as a runnable test case
`@Before`	Runs before **each** test (set-up, create objects)
`@After`	Runs after **each** test (clean-up, close database)
`@BeforeClass` / `@AfterClass`	Run once before / after all tests in the class
`@Ignore`	Skip this test

Common assertions

Assertion	Checks that
`assertEquals(expected, actual)`	Two values are equal (**expected first**)
`assertNotEquals(a, b)` **	Two values are different
`assertTrue(condition)`	Condition is true
`assertFalse(condition)` **	Condition is false
`assertNull(value)`	Value is null
`assertNotNull(value)` **	Value is not null
`assertThrows(Exception::class.java) { ... }`	The code throws that exception

Lecture example (login logic):

```
@Test
fun login_withValidCredentials_succeeds() {
    val result = authManager.login("user@test.com", "pass123")
    assertTrue(result.isSuccess)
    assertEquals("user@test.com", result.user?.email)
}
```

Regression: if someone changes `add` to return `a + b + 1`, the test fails within seconds, without anyone opening the app. That is how unit tests catch regressions (old features breaking after new changes).

Good unit tests are **fast, isolated, repeatable (deterministic) and test one behaviour each**.

4. Mocking dependencies with Mockito

The problem

A class often depends on something else (a repository, network service or database). You cannot unit test it honestly if that dependency hits a **real network or database**: it would be slow, unpredictable and may not even be available.

The solution

Mockito creates **mock objects**: fake versions of dependencies with **controlled, predictable behaviour**. Mocks **replace real dependencies** and **isolate the class under test** from external systems.

Term	Meaning	Code
Mock	A fake object created from a class/interface	`mock(UserRepository::class.java)`
Stub	Tell the mock what to return when called	`` `when`(mock.getUser(1)).thenReturn(User(1, "Nimal")) ``
Verify	Check that a method on the mock **was called** (with those arguments)	`verify(mock).getUser(1)`

`when` is a reserved word in Kotlin, so it is written with **backticks**: `` `when` ``.

Complete example (lecture + worksheet). Note: the mock is passed INTO the class being tested

```
// Production code
data class User(val id: Int, val name: String)

interface UserRepository {
    fun getUser(id: Int): User
}

class UserViewModel(private val repository: UserRepository) {   // dependency passed in
    var userName: String? = null
        private set
    fun loadUser(id: Int) {
        userName = repository.getUser(id).name
    }
}
```

```
// src/test: test the ViewModel, NOT the mock
import org.junit.Assert.assertEquals
import org.junit.Test
import org.mockito.Mockito.*

class UserViewModelTest {
    @Test
    fun loadUser_setsUserName() {
        // Arrange: create a fake repository and stub it
        val mockRepository = mock(UserRepository::class.java)
        `when`(mockRepository.getUser(1)).thenReturn(User(1, "Nimal"))
        val viewModel = UserViewModel(mockRepository)       // inject the mock

        // Act: run the code under test
        viewModel.loadUser(1)

        // Assert
        verify(mockRepository).getUser(1)                  // was the repository called with id 1?
        assertEquals("Nimal", viewModel.userName)          // did the ViewModel produce the right
result?
    }
}
```

What each line checks:

- `` `when`(...).thenReturn(...) ``: **stubbing**. When `getUser(1)` is called, the mock returns `User(1, "Nimal")` instead of going to a real database or network.
- `verify(...)`: checks the **interaction**, that the ViewModel actually **called** the repository with the right argument.
- `assertEquals(...)`: checks the **result/state**, that the ViewModel correctly set `userName`.

Common mistake: creating a mock, stubbing it, then asserting on the mock itself (`assertEquals("Nimal", mockRepository.getUser(1))`). That only tests Mockito, not your code. Always pass the mock into the real class and assert on **that class's** output.

Other verify options **: `verify(mock, times(2)).getUser(1)`, `verify(mock, never()).deleteUser(any())`.

Advantages of mocking: fast, no network or database needed, predictable results, can simulate errors ** (`thenThrow(IOException())`), tests one class in isolation. **Limitation:** it cannot tell you whether the **real** database or network actually works. That needs integration tests or testing on a device.

5. UI testing with Espresso

UI tests verify **what the user actually sees and interacts with**. They run on an **emulator or real device** (instrumented tests in `src/androidTest`). **Espresso** is Android's standard UI testing framework. It automatically waits for the UI to be idle before acting, which makes tests reliable.

The three steps of every Espresso statement

Step	Espresso call	Meaning
Find	`onView(withId(R.id.loginButton))`	Locate a view on screen (by ID or text)
Act	`.perform(click())`	Do something: click, type, scroll
Assert / Check	`.check(matches(isDisplayed()))`	Verify the view's resulting state

Pattern: `onView(ViewMatcher).perform(ViewAction).check(ViewAssertion)`

ViewMatchers (find) **	ViewActions (act)	ViewAssertions (check)
`withId(R.id.x)`	`click()`	`matches(isDisplayed())`
`withText("Login")`	`typeText("abc")`	`matches(withText("Saved"))`
`withHint("Email")`	`replaceText("abc")`	`matches(not(isDisplayed()))`
`isDisplayed()`	`clearText()`	`matches(isEnabled())`
	`closeSoftKeyboard()`	`doesNotExist()`
	`scrollTo()`, `swipeLeft()`, `longClick()`	

Complete Espresso test class (worksheet code, TODO completed)

Scenario: LoginActivity with `emailInput`, `passwordInput`, `loginButton`, and `errorText` that shows “Invalid email” when the email has no @ or a field is empty.

```
// src/androidTest/.../LoginActivityTest.kt
import androidx.test.espresso.Espresso.onView
import androidx.test.espresso.action.ViewActions.*
import androidx.test.espresso.assertion.ViewAssertions.matches
import androidx.test.espresso.matcher.ViewMatchers.*
import androidx.test.ext.junit.rules.ActivityScenarioRule
import androidx.test.ext.junit.runners.AndroidJUnit4
import org.hamcrest.Matchers.not
import org.junit.Rule
import org.junit.Test
import org.junit.runner.RunWith

@RunWith(AndroidJUnit4::class)                // run with the Android test runner
class LoginActivityTest {

    @get:Rule                                 // launches LoginActivity before each test
    val activityRule = ActivityScenarioRule(LoginActivity::class.java)

    @Test
    fun login_withInvalidEmail_showsError() {
        onView(withId(R.id.emailInput))
            .perform(typeText("not-an-email"), closeSoftKeyboard())
        onView(withId(R.id.passwordInput))
            .perform(typeText("pass123"), closeSoftKeyboard())
        onView(withId(R.id.loginButton)).perform(click())

        onView(withId(R.id.errorText))
            .check(matches(isDisplayed()))                    // error is visible
        onView(withId(R.id.errorText))
            .check(matches(withText("Invalid email")))        // with the right message
    }

    @Test
    fun login_withValidInput_hidesError() {
        onView(withId(R.id.emailInput))
            .perform(typeText("user@test.com"), closeSoftKeyboard())
        onView(withId(R.id.passwordInput))
            .perform(typeText("password123"), closeSoftKeyboard())
        onView(withId(R.id.loginButton)).perform(click())

        onView(withId(R.id.errorText)).check(matches(not(isDisplayed())))
    }
}
```

Part	Meaning
`@RunWith(AndroidJUnit4::class)`	Uses the AndroidX test runner so the test runs on a device/emulator
`@get:Rule ActivityScenarioRule(...)`	Automatically **launches the Activity** before each test and closes it after
`closeSoftKeyboard()`	Hides the keyboard so it does not cover the next view
`@Test`	Marks the method as a runnable test case
`onView`	Locates the target UI element
`matches`	Confirms the expected UI state

Tip **: turn off device animations (Developer options) to make Espresso tests more stable.

Why is a UI test much slower than a unit test?

The unit test runs directly on the JVM in milliseconds. The Espresso test must build and install the app (and test APK) on an emulator/device, launch the Activity, render the UI and simulate real input, which takes seconds.

Unit test vs UI test

Unit test (JUnit)	UI test (Espresso)
Tests logic of one method/class	Tests the screen as the user experiences it
Runs on JVM, `src/test`	Runs on emulator/device, `src/androidTest`
Milliseconds	Seconds
No Android framework	Real Android framework and UI
Mockito replaces dependencies	Real app running

6. Emulators, AVD Manager and real devices

Android Virtual Device (AVD) and the AVD Manager

An **emulator** is software on your computer that imitates an Android phone. An **AVD (Android Virtual Device)** is the **configuration** that defines one emulated device. The **AVD Manager** (called **Device Manager** in newer Android Studio) is where you **create, edit, start and delete** AVDs.

AVD definition = System Image + Virtual Hardware Profile. Lecture example configuration: **API level 34, 6.1-inch screen, 2 GB RAM**.

AVD setting	Meaning
Hardware profile	Device model, screen size, resolution, density, RAM
System image	Android version (API level) installed on the virtual device
Other	Orientation, storage, camera, network speed

Create an AVD **: Tools → Device Manager → **Create Device** → choose hardware (e.g. Pixel, 6.1") → choose **system image** (API 34) → set name/options → **Finish** → press **Run** (play) to boot it.

Emulator extra features: Extended Controls let you **simulate location (mock GPS)**, routes, battery level, network speed, phone calls, rotation and sensors.

Why emulators are needed

- **No physical hardware needed**; free.
- **Fast to create many configurations** (different screen sizes, Android versions, RAM) to test fragmentation.
- Quick iteration during daily development; easy to reset.
- Can **simulate conditions** that are hard to create on a real phone (fake GPS location, low battery, slow network).
- Run automated tests repeatably.

Why real devices are still needed

- **Genuine hardware and OS behaviour** (manufacturer customisations such as Samsung or Xiaomi skins).
- **Accurate performance and battery** testing; a low-end phone may lag where the emulator (using a powerful computer) was smooth.
- Real **sensors, camera, GPS, Bluetooth, touch feel** and real network conditions.

Connect a real device **: Settings → About phone → tap **Build number 7 times** (enables Developer options) → Developer options → turn on **USB debugging** → connect with USB → accept the "Allow USB debugging" prompt → select the device in Android Studio and Run. (Wireless debugging is also possible.)

Emulator	Real device
Runs on the development machine	Physical phone
Fast to create multiple configurations	Limited to devices you own
No hardware cost	Costs money
Can simulate GPS, battery, network	Real sensors, camera, GPS
Not accurate for performance/battery	Accurate performance and battery
Good for daily development	Essential **before release**

Lecture rule: use **emulators for daily development and quick iteration**; **validate on a real device before release**. A real device becomes essential when testing performance, hardware features (camera, sensors, GPS) and final release

checks.

7. Debugging tools in Android Studio

Tool	Question it answers	What it does
Logcat	What did the app report?	Streams system and app **log messages** and **crash stack traces** in real time
Debugger	What do the variables actually hold?	**Pauses** execution at breakpoints to inspect variables and the call stack
Layout Inspector	What does the view hierarchy really look like?	Examines the **live view hierarchy** and layout properties of the running app
Profiler	What is the app costing?	Measures **CPU, memory, network** (and energy) usage over time

Logcat

Logcat shows messages from the system and from your code. When the app crashes, Logcat shows the **exception and stack trace** (the line that caused the crash).

```
class LoginActivity : AppCompatActivity() {
    private fun login(email: String, password: String) {
        Log.d("LoginActivity", "Attempting login for $email")       // Debug
        if (isValid(email, password)) {
            Log.i("LoginActivity", "Login succeeded")                // Info
        } else {
            Log.e("LoginActivity", "Login failed: invalid input")    // Error
        }
    }
}
```

`Log.x(TAG, message)`: the **tag** identifies where the message came from (usually the class name), so you can **filter** Logcat by it. `Log.e(TAG, "msg", exception)` also prints the exception's stack trace.

Level	Method	Letter	When to use
Verbose	`Log.v()`	V	Very detailed tracing; lowest priority
Debug	`Log.d()`	D	Debugging info during development
Info	`Log.i()`	I	Normal important events (login succeeded)
Warn	`Log.w()`	W	Something unexpected but not an error (retrying)
Error	`Log.e()`	E	Errors and failures
Assert **	`Log.wtf()`	A	"What a terrible failure": should never happen

Filter Logcat by **tag**, **package/process**, **log level** or text, for example `tag:LoginActivity level:error`.

Never log passwords, tokens or private user data, even at Debug level: logs can be read by other tools, saved in bug reports, and may reach production builds.

Debugger

- A **breakpoint** pauses the app at a specific line (click in the left margin next to the line number).
- Run with **Debug** (bug icon) instead of Run.
- While paused you can inspect **variable values**, the **call stack** and object state, and add **watches**.
- **Conditional breakpoint:** right-click the breakpoint and enter a condition; it pauses **only when the condition is true**, for example `email.isEmpty()`.
- Works on emulators and real devices connected via USB.

Control	Action
Step Over	Run the current line and go to the next line **without entering** called functions
Step Into	**Go inside** the function called on the current line
Step Out	Finish the current function and return to the caller
Resume **	Continue running until the next breakpoint

Layout Inspector and Profiler

- **Layout Inspector:** shows the live view tree, each view's size, position, padding and attributes. Use it when a view overlaps, is invisible, or is in the wrong place.
- **Profiler:** CPU (slow code, laggy scrolling), Memory (leaks, growing memory), Network (requests and data sizes), Energy (battery).

A screen feels slow to scroll: which two tools? The **Profiler** (CPU and memory, to see what work is slowing it) and the **Layout Inspector** (to see if the view hierarchy is too deep or complex).

8. Typical debugging workflow (lecture, 8 steps)

1. **Reproduce** the bug → 2. **Add a breakpoint** → 3. **Run in debug mode** → 4. **Hit the breakpoint**, app pauses → 5. **Inspect variables** → 6. **Form a hypothesis** → 7. **Apply a fix** → 8. **Re-run tests** to confirm (and check nothing else broke).

Worked example (worksheet planted bug):

```
private fun isValid(email: String, password: String): Boolean {
    if (email.isEmpty() || password.isEmpty()) return false
    if (email.contains("@") && password.length > 6) return false   // bug
    return true
}
```

Reproduce: enter a valid email and a 7+ character password; the error still shows. Breakpoint on the first line, run in debug, **Step Over** each line: line 2 passes, line 3 returns `false` for valid input. Hypothesis: the condition is inverted. Fix:

```
private fun isValid(email: String, password: String): Boolean {
    if (email.isEmpty() || password.isEmpty()) return false
    return email.contains("@") && password.length > 6
}
```

Re-run the Espresso tests to confirm they pass.

9. Which tool or test? (scenario table)

Scenario	Best choice
A tax/total calculation returns the wrong value	**JUnit** local unit test
A class depends on an API client or database that is hard to control	**Mockito** mock
Tap a button and check that new text appears / screen changes	**Espresso** UI test
App crashes when a screen opens	**Logcat** (stack trace) + **Debugger**
Need to see a variable's value at a certain line	**Debugger** with a breakpoint
A view overlaps or disappears	**Layout Inspector** + test on different screen sizes
Memory grows while scrolling / screen lags	**Profiler**
Works on emulator, but GPS/camera fails	Test on a **real device**; check permissions and sensor state
Need to test many Android versions/screen sizes quickly	Several **AVDs** in the AVD Manager
Test needs a real Activity	Put it in **`src/androidTest`** (instrumented)

10. Practice questions with short model answers

Difference between a unit test and a UI test? A unit test checks one function's logic on the JVM; a UI test checks what the user sees by simulating taps and typing on an emulator or device.

Why do unit tests run faster than UI tests? They run directly on the JVM without installing the app, launching a device or rendering any UI.

What role does Mockito play in a unit test? It replaces real dependencies (network, database) with controlled fake objects so the class under test can be tested in isolation, quickly and predictably.

What does `when`(...).thenReturn(...) do? It stubs the mock: when that method is called with those arguments, it returns the given value.

`verify` vs `assertEquals` in a Mockito test? `verify` checks that a method was called on the mock; `assertEquals` checks the result produced by the class under test.

Name the three parts of an Espresso test. onView (find the view), perform (act on it), check (assert the result).

Write an Espresso line to click a button with ID btnSave. `onView(withId(R.id.btnSave)).perform(click())`

Write an Espresso line to check that "Saved" is shown in tvStatus.
`onView(withId(R.id.tvStatus)).check(matches(withText("Saved")))`

A test needs a real Activity. Which folder? `src/androidTest`, because it is an instrumented test that needs the Android framework on a device or emulator.

Give one reason to test on a real device instead of only an emulator. Real devices show true performance, battery use and hardware behaviour (camera, sensors, GPS) that an emulator cannot fully reproduce.

Why are emulators needed? They let developers test many screen sizes and Android versions quickly, without buying devices, and simulate conditions like GPS location.

What is the AVD Manager? The Android Studio tool used to create and manage virtual devices (hardware profile + system image).

What does Logcat show? Real-time system and app log messages, warnings, errors and crash stack traces, which can be filtered by tag and level.

Why should logs never contain a password? Logs can be read by other tools or included in reports, exposing sensitive user data.

Step Over vs Step Into? Step Over runs the current line without entering called functions; Step Into goes inside the called function.

Name two tools to diagnose a slow screen. Profiler and Layout Inspector.

A test passes on your machine but fails on a classmate's. Two possible causes? Different emulator/device configuration (API level, screen size) or different environment (dependency versions, animations on, network or test data).

Write the JUnit assertion for "result should be 10". `assertEquals(10, result)`